

24 April 2024

Code Generation from Flowchart using Optical Character Recognition & Large Language Model

Aryaman Darda, Reetu Jain

Abstract

Students find it difficult to learn the syntax of programming languages when they are new to coding. Some students get discouraged and even quit their coding journey because of having trouble with memorizing all the syntactic rules of a programming language. Flowcharts are an effective way to help students visualize the logic of an algorithm developed to solve a problem. But some students find it challenging to convert their flowcharts into executable code and even when they do, some get frustrated because of the errors they encounter while running the code they have written. So students who are not well versed with the syntax of a programming language can face difficulty debugging their programs even when they understand the logic behind the algorithms. This research aimed at developing a tool that can assist young students with less coding experience to learn programming. An intuitive user interface for the application was developed using a python library called gradio. The functionality of taking flowchart images as user inputs and providing equivalent python code with a brief explanation was incorporated on the UI. Text from the uploaded flowchart image is extracted in 3 steps which include feature extraction using deep learning models like ResNet and VGG, sequence labelling using Long Short-Term Memory networks and decoding the sequence labels using Connectionist Temporal Classification algorithms. The extracted text is then provided to an open source large language model with 70 billion parameters called Llama-2-Chat for the generation of the python code. The generated python code is displayed on the gradio UI along with a brief explanation of the code. The user can copy the code and run it on the IDE of their choice. The application was evaluated with a test dataset of several flowchart images. The generated python codes for the test flowchart images were executed in a code editor and the produced outputs were found to be correct. The generated code got executed without any errors for 75 % of the flowchart images from the test dataset.

Keywords

automated code generation, computing and processing, large language models (llm), llama 2, optical character recognition (ocr), python, structured flowcharts

Code Generation from Flowchart using Optical Character Recognition & Large Language Model

Aryaman Darda¹ and Reetu Jain¹

¹Affiliation not available

April 24, 2024

Abstract

Students find it difficult to learn the syntax of programming languages when they are new to coding. Some students get discouraged and even quit their coding journey because of having trouble with memorizing all the syntactic rules of a programming language. Flowcharts are an effective way to help students visualize the logic of an algorithm developed to solve a problem. But some students find it challenging to convert their flowcharts into executable code and even when they do, some get frustrated because of the errors they encounter while running the code they have written. So students who are not well versed with the syntax of a programming language can face difficulty debugging their programs even when they understand the logic behind the algorithms. This research aimed at developing a tool that can assist young students with less coding experience to learn programming. An intuitive user interface for the application was developed using a python library called gradio. The functionality of taking flowchart images as user inputs and providing equivalent python code with a brief explanation was incorporated on the UI. Text from the uploaded flowchart image is extracted in 3 steps which include feature extraction using deep learning models like ResNet and VGG, sequence labelling using Long Short-Term Memory networks and decoding the sequence labels using Connectionist Temporal Classification algorithms. The extracted text is then provided to an open source large language model with 70 billion parameters called Llama-2-Chat for the generation of the python code. The generated python code is displayed on the gradio UI along with a brief explanation of the code. The user can copy the code and run it on the IDE of their choice. The application was evaluated with a test dataset of several flowchart images. The generated python codes for the test flowchart images were executed in a code editor and the produced outputs were found to be correct. The generated code got executed without any errors for 75 % of the flowchart images from the test dataset.

Introduction

Coding, one of the most important skills to learn in the 21st century, helps children to sharpen their problem solving skills and boosts their creativity. It allows programmers to translate commands into binary codes using different programming languages such as Java, C++ and Python. The fast pace of digitisation has made children familiar with technology and the Internet. From an early age, students are exposed to technological advancements via the smartphones they use. Learning the techniques behind these advancements can effectively redirect their interest in these devices towards constructive pursuits, making their screen time more purposeful. It can help them become creators of technology rather than just consumers. Children may initially find coding difficult, as each programming language has its distinct syntax, which children have to learn to write code correctly. Acquiring proficiency in the syntactical rules of a specific language takes time, and encountering errors is inevitable during the journey of learning to code. Debugging can also be frustrating and it may discourage children from learning coding. The objective of this project is to create an application that can generate the code from the image of a flowchart given to it. Students can upload an image of their flowchart on the user interface and the application will generate the python code for the uploaded flowchart image. Python's simplicity and readability make it an ideal language for beginners. The application assists students to convert the flowchart of their solutions into executable python code effortlessly.

The generated code can help students see how their solution works after implementation without having to become well-versed in the syntax of the programming language they choose to implement it in. To develop the application, various computer vision techniques and large language models were used. A computer vision technique called Optical Character Recognition was used for extracting the textual content from the flowchart blocks in the uploaded image. A large language model named “Llama 2” with 70 billion parameters was used to transform the textual information extracted from the flowchart into its corresponding python code. By automating the process of code generation from flowchart images, the application speeds up the process of implementing the logic figured out to solve a problem. It can also be used to enhance educational tools used for teaching coding to younger students.

Literature Review

In the study conducted by **R. Leupers et al. [1]**, the importance of software in digital signal processing (DSP) applications was examined, highlighting the need for automated tools to support DSP based software development. Techniques for high-level block-diagram-based modeling of DSP applications and translating block-diagram specifications into efficient C programs through global target-independent optimization methods, and compiling C programs into optimized machine code for programmable DSP processors were reviewed. **Ansong Ni et al. [2]** introduced LEVER, a straightforward approach to enhance language-to-code generation by training verifiers to assess program correctness based on natural language input, the program itself, and execution results. The sampled programs were ranked by integrating the verification score with the LLM generation probability. LEVER achieved notable improvements ranging from 4.6% to 10.9% with code-davinci-002 across four datasets containing samples from table QA, math QA, and fundamental Python programming domain. **Chanchai Supaartagorn et al. [3]** introduced an automatic code generator tool based on structured flowcharts, comprising basic shapes that can be combined to form structured flowcharts convertible into source codes. The tool’s performance was evaluated with two groups: 5 experts and 93 general users. Results demonstrated high satisfaction levels among both groups, with average values of 4.48 and 4.27 and standard deviations at 0.59 and 0.64 for experts and general users respectively. The tool demonstrated agreeable performance. **Tomasz Szydło et al. [4]** noticed that programming libraries often demand excessive resources, making them impractical for deployment on embedded processors. With their research they introduced the concept of source code generation for machine learning models, along with algorithms for generating commonly utilized machine learning methods. The effectiveness of this concept has been validated through various use cases.

Batuhan Aşıroğlu et al. [5] observed that the web design process starts with creating mock-ups for individual web pages, either manually or using graphic design tools. These mock-ups are then converted into structured HTML or similar code by software engineers, undergoing refinements until the desired template is achieved. Their research aimed to automate the code generation process from hand-drawn mock-ups, using computer vision techniques and incorporating select deep learning methods. Their system demonstrated a method accuracy of 96% and a validation accuracy of 73%. **Samantha Ray et al. [6]** observed that the existing user interface driven solutions for creating flowcharts present challenges to learners with intricate drag-and-drop menus, while sketching-based alternatives lack support beyond initial pseudocode generation. The researchers created Flow2Code to facilitate the translation of hand-drawn flowcharts into code. Flow2Code recognizes various flowchart shapes, and then converts the flowcharts into executable code. It has an intuitive, interactive interface for users to modify both their flowchart and resulting code. Humans have the ability to comprehend technical documents effortlessly. Researchers have made several attempts to impart such comprehension capabilities to AI-based systems. Research done by **N. G. Bourbakis et al. [7]** focused on the utilization of the interaction between two technical document (TD) modalities: block diagrams and associated natural language text to develop a system for generating pseudocode that defines the functionality of the system under examination automatically. The methodology involves mapping the TD modalities into Stochastic Petri-nets (SPN) to enhance system diagrams that are used for pseudocode generation. With this method the researchers aimed to achieve automatic deep comprehension of technical

documents. Aspects like the use of diagram images and the automated understanding of mathematical formulas in technical documents remain relatively understudied. **Gkorgkolis Nikolaos et al. [8]** introduced a new formal scheme for modeling digital diagram images, extending to a generative framework for creating artificial images and annotations. They proposed a method to convert the pseudocode generation problem into an image captioning task, employing a range of techniques based on adaptive image partitioning. They addressed semantic understanding of mathematical formulas by conducting an evaluative survey, which was followed by the introduction of a formal synthesis framework that utilized formula graphs as metadata to produce valuable formulas. This synthesis framework is validated using a deep geometric learning mechanism that utilizes formula data to simulate missing a priori knowledge.

Enrique Dehaerne et al. [9] analyzed 37 publications sourced from the arXiv and IEEE Xplore databases, which were based on projects in which ML models were trained on programming language data to produce code. They identified three main paradigms of code generation: description-to-code, code-to-description, and code-to-code. These papers primarily focused on ML applications such as generating code from natural language descriptions, documentation generation, and automatic program repair. Commonly used ML models for these research projects include recurrent neural networks, transformers, and convolutional neural networks, along with various other neural network architectures and non-neural techniques. Comparisons of model types, tokenizers, data volume and quality, and evaluation methods for synthesized code were also discussed in this review. Presently, researchers are focused on generating code from requirement documents; however, existing methods often struggle with requirements that demand intricate problem-solving abilities. **Zejie Liu et al. [10]** introduced a novel method for generating source code from flowcharts along with textual descriptions. The researchers manually curated a benchmark dataset comprising 320 flowcharts paired with their source codes. Adapting existing approaches to this new task has its challenges due to the distinctive nature of flowcharts containing various elements and the multiple connections between nodes within them. To address these challenges, The researchers have proposed a two-stage code generation model. In the first stage, a structure recognition algorithm is used to translate the flowchart into pseudo code. In the second stage, a code generation model converts the pseudo-code into executable code. To ensure a comprehensive understanding of algorithms, it is necessary to devise methods for generating corresponding text descriptions. The study conducted by **Sagarika Ghosh et al. [11]** aligns algorithms in various forms, such as pseudocode and hand-drawn flowcharts, with textual explanation. The researchers proposed rules for generating pseudocode from hand-drawn flowcharts and a transfer learning method based on S-DistilBERT to find the similarity score between different forms of algorithms and their text descriptions. Block and line identification, along with OCR were used to generate pseudo codes from hand-drawn flowcharts. Experimental results indicate an 85% success rate in generating equivalent pseudocode. Their fine-tuned S-DistilBERT model achieved accuracies of 75.59% for matching existing pseudocode and 74.57% for generated pseudocode with their corresponding textual descriptions. The rules devised by the researchers have been found to be suitable only for non-recursive flowcharts. In the research done by **Xiang-Hu Wu et al. [12]**, they proposed a structure identification algorithm for structured flowcharts, verified for correctness using enumeration iteration. An automatic code generation algorithm was also introduced, which was validated through enumeration iteration. Finally, an integrated development platform was developed utilizing these algorithms, and incorporating flowchart modeling, code automatic generation, and support for CDT\GCC\GDB. The effectiveness of the proposed algorithms were evaluated through practical implementation.

METHODOLOGY

The proposed methodology involves the use of Optical Character Recognition (OCR) and Large Language Model (LLM) for creating an application that generates python codes from flowchart images. Flowchart images uploaded on the application's user interface that is developed using Gradio are processed using a python library called EasyOCR to extract the textual content from the flowcharts. It uses deep learning models like VGG and ResNet for feature extraction, Long Short-Term Memory networks to understand the sequential context of the extracted features and Connectionist Temporal Classification algorithm to decode

the labeled sequences into actual text. A query is constructed using the text extracted with OCR that is provided as a prompt to an LLM called Llama 2 Chat (70 Billion Parameters Version) accessed using an API from an online platform called Replicate. The output returned by the LLM contains the python code and a brief explanation of it which is extracted and stored in a python string object before returning it to the user interface as the output for the uploaded flowchart. Functionality to copy the python code displayed on the UI was also incorporated to allow the user to test the code generated for the uploaded flowchart.

Text Extraction using Optical Character Recognition

Optical Character Recognition (OCR) is a process in which readable text is extracted from images containing textual content. It helps to interpret printed or handwritten text from various sources such as scanned documents or captured images. OCR operates on the principles of pattern recognition and feature extraction. First, an input image containing text is preprocessed by reducing the noise using Gaussian filters and Morphological operations like Erosion and Dilation to enhance its quality. Then the processed image is segmented into individual characters to isolate them from the background employing techniques like text line extraction with hough transform and word extraction with connected component analysis. Statistical features like density and direction of the foreground pixels in the image are extracted using zoning, projection histograms, etc. Structural features like cross points, strokes, loops, horizontal curves, etc are also extracted. Finally, pattern matching algorithms are utilized to interpret the extracted features, generating readable text output. A library called EasyOCR is being used for extracting the text from the flowchart components in the uploaded image. The uploaded flowchart image is converted into an array using OpenCV. The image array is provided as an argument to the readtext method of the Reader class from the EasyOCR library. The method returns the extracted text, coordinates of the top left and bottom right corner of the bounding box and the confidence score of every word extracted from the image. All the extracted data is stored in a variable. The bounding boxes of the extracted text are drawn on the uploaded image and the text is put on the uploaded image at appropriate positions using OpenCV methods as shown in Fig 1. Using Matplotlib, the flowchart image with the bounding boxes and the extracted text are displayed in a window.

Fig 1. Extraction of text from flowchart components in the image using OCR

Creating Structured Query for Large Language Model (LLM)

A custom python function named `create_query` was created to generate a structured query for the Llama 2 LLM to generate the Python code from the extracted text. The created function takes two parameters as inputs which are as follows : A variable which contains the textual content and the other details extracted from the flowchart image. This input is an iterable containing tuples where each tuple contains three elements which are the bounding box coordinates, the extracted text, and the confidence score for that text. The other argument provided to the function is a python string containing "#". The function uses a for loop to iterate all elements in `img_text` and extracts only the text from it, this text is appended to the python string and then the "def" string is attached to it at the end. The final structured query string is returned from the function. The query string containing the OCR extracted text can be provided to a LLM as a user input and it can generate the python code for the extracted text as shown in Fig 2.

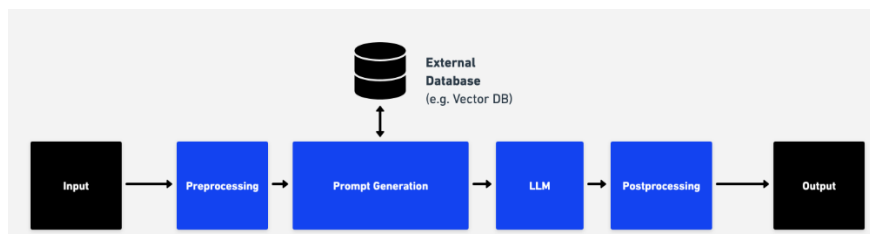


Fig 2. A forward pass through an LLM to produce an output from the user input

Creation of Replicate API Token

An Application Programming Interface (API) serves as an intermediary between different software systems, allowing them to communicate and interact with each other. An API defines the methods and protocols that developers can use to access the functionality and data of some other application or an online platform. An API token, also called an API key or authentication token, is a unique and secure piece of data used to authenticate and authorize access to an API. It acts as a credential that allows a developer or an application to interact with an API in a secure manner. An API token was generated for accessing Large Language models like Llama from a platform called Replicate which hosts open source machine learning models as shown in Fig 3. This API token was further used to employ Llama 2 LLM with 70 billion parameters for generating the python code from the textual content extracted from the uploaded flowchart image.

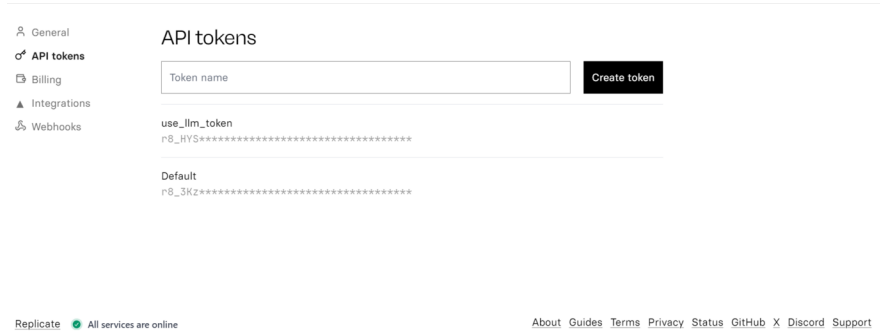
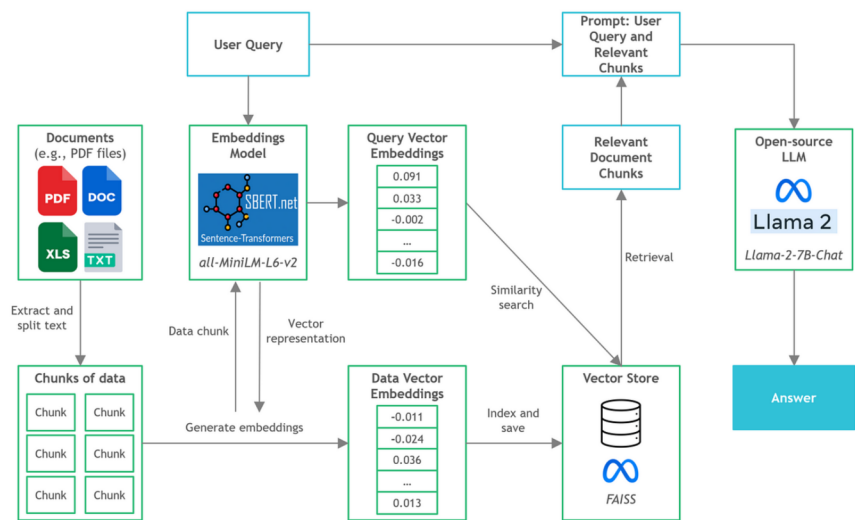


Fig 3. Generation of Replicate API Token

Use of Llama2 LLM through Replicate API Token



A large language model (LLM) is a type of Machine Learning model that utilizes a Neural Network architecture. A LLM is designed to understand and generate natural language text. A LLM is trained on large

amounts of text data to make it learn the complexities of language patterns, grammar rules, and semantic relationships. Llama 2 is an open source generative AI large language model available in 3 sizes with 7 Billion, 13 Billion and 17 Billion parameters. All the Llama models are available on the Hugging Face platform which is a website which hosts open source machine learning models. NousResearch evaluated the Llama-2-70B model using four A100 40GB GPU setups with two prompt lengths: 1500 input tokens and 100 output tokens, and 50 input tokens and 500 output tokens. The assessment revealed the model's capabilities, with latency best at 7.4 seconds and 33 seconds for the respective configurations. Throughput peaked at 1.1 RPS for the former and 0.8 RPS for the latter. All the new LLM advancements like Ghost Attention, In-Context Temperature re-scaling, Grouper query attention, etc have been incorporated in Llama 2. Its tokenizer uses a byte pair encoding algorithm. The standard transformer architecture, with pre-normalization using RMSNorm, and rotary positional embedding has been utilized in Llama 2 LLM. The model performs well on several tasks like coding, commonsense reasoning as well as Q&A tasks. The Llama 2 Chat version has also outperformed other open source models by a good margin. The working of the Llama 2 LLM has been described through a flowchart as in Fig4. An environment variable named "REPLICATE_API_TOKEN" was created to hold the Replicate AI's API token. This token is used for authentication purposes when requests are being made to the Replicate API from this application. A Replicate client object was created to which the variable containing the API token was provided. The Llama 2 Chat model with 70 billion parameters is called using this client object and the created query for generating python code is provided to it as a prompt. The output returned by the Llama 2 model is stored in a python object that can be iterated.

Fig 4. Detailed Working of Llama 2 Chat LLM

Extraction of Structured Python Code from Llama2 LLM Output

The python object that has the output code generated by the Llama 2 70B Chat LLM is iterated using a loop construct and all the python code from the Llama output is stored in a variable as a string.

Integration of User Interface with OCR and LLM

Functions created to extract the text from the flowchart image, to create the structured query for the LLM, to send the structured query as a prompt to the Llama 2 Chat LLM and extract the python code and its explanation from the LLM output were put into a single python module that was used with the User Interface code. When the user uploads an image of a flowchart on the UI, the "generate" function that we have created is called and the uploaded flowchart image is provided to the function as an input, then the function extracts the path of the uploaded image and then makes a call to the read_image function from the custom utils module which converts the image into an array. This image array is provided to the detect_text function to extract the text from the flowchart. The extracted flowchart text is converted into a structured python code generation query using the create_query function which is used as a prompt for the Llama 2 LLM. The Llama LLM is accessed using the replicate API token and then it is prompted with the code generation query. The output containing the python code for the flowchart and its brief explanation produced by the Llama 2 LLM is stored in a python object which is iterated to extract the python code and its explanation. The python code and its explanation is appended to a blank string using the python_code_string function and the python code for the uploaded flowchart and its explanation is returned as a string to the user interface. The flowchart image and its equivalent python code are displayed as output on the user interface. The users can copy the generated python code and execute it on the Integrated development environment or a code editor of their choice.

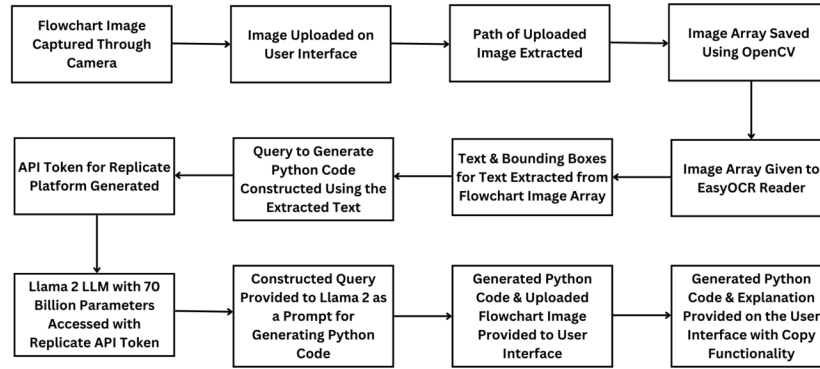


Fig 5. Python Code Generation from UI using Llama2-70B-Chat LLM

IV. Results

To achieve the objective of this research, a methodology involving pre-processing the uploaded flowchart image, text extraction from the flowchart image using OCR, code generation with large language model inference was implemented. A diverse test dataset of flowchart images representing various programming tasks was created. This dataset was used for evaluating the created application. Python codes generated for the test flowchart images were executed in a code editor and the outputs produced were checked. Quality of the Python codes generated for the test flowchart images was evaluated for their accuracy, syntactic correctness, and adherence to best programming practices. Codes generated for 75% of the flowchart images from the test dataset executed without any compile time and runtime errors. Best programming practices were followed in 64% of the generated python codes for test flowcharts. The experiments demonstrated promising results, with the application successfully generating Python code and its brief explanation from flowchart images with a high degree of accuracy. It was observed that the performance of the application varied depending on the complexity of the flowchart and the clarity of the flowchart images. However, even for 62% of challenging cases where uploaded flowchart images were not clear and the text in the flowchart components wasn't clearly visible, The application was able to generate the correct python codes.

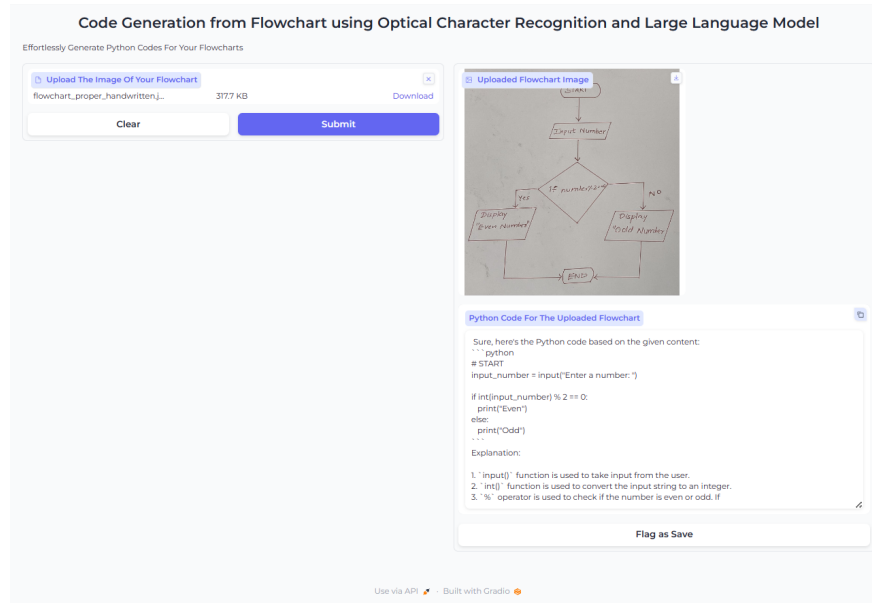


Fig 6. Test Flowchart Image Uploaded on the UI and its Python Code Generated and Displayed as the Output

V. Conclusion

In conclusion, This research demonstrates the feasibility of translating visual representations of algorithms into executable code. The implications of this study is significant, as it offers potential benefits to students, developers and educators. Students can benefit from automated code generation as a learning aid, reinforcing their understanding of programming syntax. Educators can utilize this tool to teach programming to young students in an easy manner, making coding accessible for all students regardless of their coding experience. Additionally, For developers, this application can streamline the process of translating flowcharts into implementation-ready code, saving time.

VI. Future Scope

While the developed tool has shown promising results, there are areas for future improvement and research. Enhancing the effectiveness of text extraction from noisy or cluttered images remains a challenge, as does improving the handling of complex control flow structures and edge cases. Additionally, exploring the applicability of the developed methodology to other programming languages and expanding its scope to support a wider range of flowchart types are avenues worth pursuing.

Acknowledgment

I would like to express my gratitude to the mentors Mrs. Reetu Jain and Mr. Suraj Sharma of On My Own Technology Pvt. Ltd. for extending their help in carrying out the research.

References

1. Domí, Eladio, Beatriz Pérez, and Ángel L. Rubio. "A systematic review of code generation proposals from state machine specifications." Information and Software Technology 54.10 (2012): 1045-1066.

2. Shimonaka, Kento, et al. "Identifying auto-generated code by using machine learning techniques." 2016 7th International Workshop on Empirical Software Engineering in Practice (IWESEP). IEEE, 2016.
3. Supaartagorn, Chanchai. "Web application for automatic code generator using a structured flowchart." 2017 8th IEEE International Conference on Software Engineering and Service Science (ICSESS). IEEE, 2017.
4. Szydło, Tomasz, Joanna Senderek, and Robert Brzoza-Woch. "Enabling machine learning on resource constrained devices by source code generation of the learned models." Computational Science–ICCS 2018: 18th International Conference, Wuxi, China, June 11-13, 2018, Proceedings, Part II 18. Springer International Publishing, 2018.
5. Aşıroğlu, Batuhan, et al. "Automatic HTML code generation from mock-up images using machine learning techniques." 2019 Scientific Meeting on Electrical-Electronics & Biomedical Engineering and Computer Science (EBBT). IEEE, 2019.
6. Ray, Samantha, et al. "Flow2Code: Transforming Hand-Drawn Flowcharts into Executable Code to Enhance Learning." Inspiring Students with Digital Ink: Impact of Pen and Touch on Education (2019): 79-103.
7. Bourbakis, Nikolaos G., Giorgia Rematska, and S. Mertoguno. "Deep Understanding of Technical Documents: Part II. Automatic Extraction of Pseudocode." International Journal on Artificial Intelligence Tools 30.03 (2021): 2150016.
8. Gkorgkolis, Nikolaos. Deep Understanding of Technical Documents: Automated Generation of Pseudocode from Digital Diagrams & Analysis/Synthesis of Mathematical Formulas. Diss. Wright State University, 2022.
9. Dehaerne, Enrique, et al. "Code generation using machine learning: A systematic review." Ieee Access (2022).
10. Liu, Zejie, et al. "Code Generation From Flowcharts with Texts: A Benchmark Dataset and An Approach." Findings of the Association for Computational Linguistics: EMNLP 2022. 2022.
11. Ghosh, Sagarika, et al. "Matching of hand-drawn flowchart, pseudocode, and english description using transfer learning." Multimedia Tools and Applications (2023): 1-29.
12. Mentari, Mustika, et al. "Automatic Java Code Generation System from Flowchart for Basic Programming Learning." Proceedings of the 5th Annual Advanced Technology, Applied Science, and Engineering Conference (ATASEC) 2023. Vol. 229. Springer Nature, 2024.